

Results Browser: Must-Have Features

Change report · branch **fixmeseb-new-results-panel** · based on 4f8a5ef fix(plugins): eliminate legacy uninstalled tracking and prevent crash on Hub install · October 02, 2026
Uncommitted working-tree changes in the ThetaIDE frontend and NeSyRL backend API

1. Summary

The Results browser (sidebar tab under Monitor) was extended with the four must-have features identified in the review of the panel. All changes are additive: no existing button, column action, menu entry or API field was removed. One earlier change on this branch (showing the Results tab by default) is included at the end of section 3 so that the report covers every uncommitted edit.

#	Feature	Problem before	What changed
1	Show runs started from the command line	The table listed only runs launched from the GUI (records in .thetaide/runs). Experiments run with run_pipeline.py or SLURM never appeared.	The backend's GET /api/runs now summarises every results/logs folder; the GUI requests it when the backend connects, after a run finishes and from a new Refresh button, and lists folders it has no record for.
2	Sort by column	Sorting was not enabled; there was no way to rank runs.	Click any header to sort. Numbers, dates and durations sort by value (not text); missing values always sort last. Default order: newest first.
3	More useful columns	Five columns: Experiment, Seed, Status, Reward (latest only), Source.	Eleven columns: Experiment, Started, Method, Environment, Seed, Timesteps, Duration, Status, Best reward, Latest reward, Source.
4	Hide example runs / filter	The two synthetic example runs were always listed beside real ones; search covered only name, seed and status.	Examples are hidden once real runs exist (a checkbox shows them). New Source and Status filters; search covers every visible column.

Files touched (git diff --stat, plus two new test files):

File	Lines (+ / -)	Role
frontend/app.py	+153 / -30	Results panel UI, table fill, disk listing, metrics loading, small fixes
frontend/model.py	+154 / -0	Pure data logic: row values, sort keys, filters, command-line run records
frontend/settings.py	+1 / -0	Default sidebar: Results visible (earlier change)
frontend/widgets.py	+23 / -1	SortableItem table cell
src/app/api/app.py	+89 / -7	GET /api/runs summaries; newest version_N chosen by number
tests/test_api.py	+62 / -0	3 new backend tests
tests/test_results_model.py	+95 (new file)	7 data-logic tests (no Qt)
tests/test_results_browser.py	+105 (new file)	6 Qt tests of the panel

2. Before and after

Before: a fresh install showing only the two synthetic examples and five columns.

Experiment history

Select one run to inspect, or two to compare (Ctrl + click).

Filter by experiment name, seed, or status...

Experiment	Seed	Status	Reward	Source
cartpole_ppo_baseline	42	example	489.2	Simulated
cartpole_seed_study	7	example	494.3	Simulated

Load saved configCompare two runsView plot

Results panel on branch HEAD (4f8a5ef) with no backend.

After: three runs started outside the app, found in results/logs through the backend.

Experiment history

Select one run to inspect, or two to compare (Ctrl + click). Click a column header to sort.

Filter by name, method, environment, seed, status, or source...

All sources

All statuses

Show example runs

Refresh

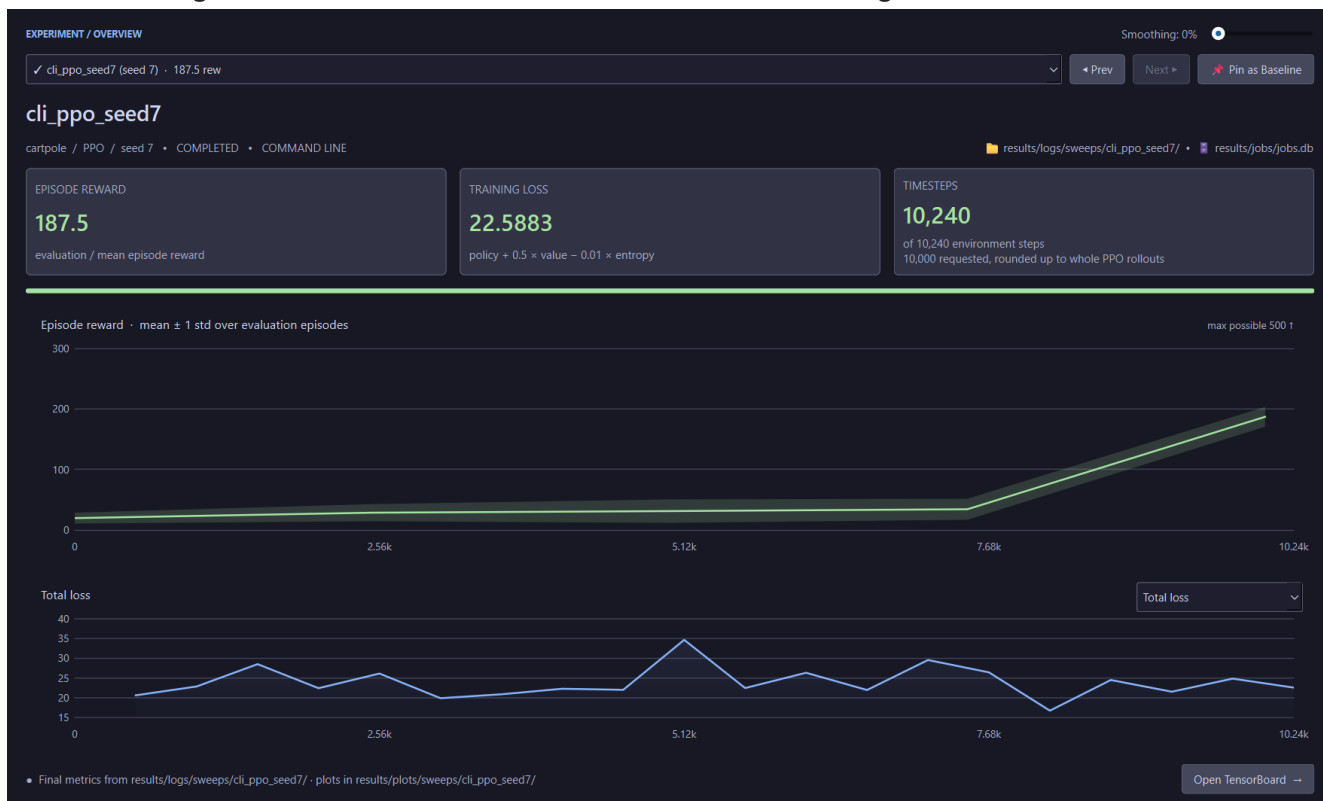
- 3 run folders in results/logs - 3 started outside the app

Experiment	Started	Method	Environment	Seed	Timesteps	Duration	Status	Best reward	Latest reward	Source
cli_minigrid_ppo	2026-09-30 08:00	ppo / dnn	minigrid	42	10,240	1 h 02 min	completed	34.8	33.7	Command line
cli_ppo_seed7	2026-09-28 05:15	ppo / dnn	cartpole	7	10,240	14.8 s	completed	187.5	187.5	Command line
cli_ppo_unfinished	2026-10-01 14:30	ppo / dnn	cartpole	42	10,240	—	incomplete	34.8	33.7	Command line

Load saved configCompare two runsView plot

Test data: copies of a real run in a temporary results folder, served by the real backend. Examples are hidden because real runs exist; cli_ppo_unfinished has no recorded end time, so its status is 'incomplete' and its duration is unknown.

After: selecting a command-line run loads its curves into the Training monitor.



The header reads `COMMAND LINE` instead of a job ID, and the caption shows the run's own environment. Values agree with the table (187.5).

3. Changes in detail

Line numbers refer to the working-tree files. Every changed region is listed exhaustively in Appendix A.

Feature 1 — Show runs started from the command line

The fix. The backend already had GET /api/runs, but it returned only each folder's group and experiment ID (its metadata field read a runtime.json that the pipeline never writes at that level, so it was always empty), and nothing in the GUI called it. The endpoint now adds a summary of each folder read from files the pipeline already writes. The GUI turns every folder it has no record for into an in-memory run record and loads that run's metrics the first time it is selected.

File	Code changed	What it does
src/app/api/app.py	_latest_version_file, _float, _agent_summary, _run_config_summary (new, lines 113–188)	Read config.yaml (environment, seed, timesteps, methods), each agent's newest runtime.json (start, end, training time) and metrics.csv (best/latest reward, timesteps).
src/app/api/app.py	list_runs (191–223)	Existing fields kept unchanged; adds config, agents and finished to each item.
src/app/api/app.py	get_metrics (226–)	Uses the shared helper to choose the newest version_N folder.
frontend/model.py	disk_run, disk_run_id, _value, run_source, run_name (149–202)	Builds a run record (origin 'cli', transient) whose config is valid for the run store and builder.
frontend/model.py	metric_points_from_api (111–122)	Parses the metrics endpoint's text values; now shared by the old backfill code and the new loader.
frontend/app.py	request_disk_runs, disk_runs_received, update_disk_status, load_disk_metrics, disk_metrics_received (2113–2176)	Requests and merges the listing; status line; lazy metrics loading; redraws a multi-run comparison when its curves arrive.
frontend/app.py	set_backend_state (1366), finish (1796), select_run (1815), table_selected (2189)	Listing requested when the backend goes offline→connected and after a trained run finishes; metrics loaded on selection and for multi-selection.
frontend/app.py	persist (1449)	Saving a listed run 'adopts' it into the run store (drops the transient flag) so it is not listed twice.
frontend/app.py	render_run (1922–), open_job (1745)	Fixes found in testing: the Monitor header and Queue → 'open job' assumed every trained run has a job_id and raised KeyError (which aborts the app) for command-line runs.
frontend/app.py	load_selected_config (2218)	Refuses (with a status message) to load a non-CartPole/PPO command-line run into the builder.

Behaviour that may have changed

- **New network requests:** the GUI now calls GET /api/runs on connect, after each finished trained run and on Refresh, and GET /api/runs/.../metrics when a command-line run is selected.
- **The run list now contains more records:** command-line runs also appear in the Monitor's run dropdown (with Prev/Next), the Plot viewer's source list, the comparison overlay and the Settings panel's 'Total run records' count.
- **Run order:** each listing re-sorts the in-memory run list newest-first by start time. Before, runs launched during a session were simply inserted at the top. The Monitor dropdown order can therefore change.
- **New status 'incomplete'** for folders with no recorded end time. It covers both runs still training outside the app and runs that crashed; the app cannot tell these apart.

- **Monitor header** for command-line runs shows 'COMMAND LINE' and the run's environment instead of the fixed 'CartPole-v1 / PPO'. App-launched and simulated runs are unchanged.
- **get_metrics now picks version_10 over version_9** (by number). Before, text order picked version_9. Only affects agents with ten or more versions; it is a correction, but it changes which file is served.
- **Reward summaries count only rows that have a step count**, exactly as the charts do. Lightning repeats each evaluation in an epoch row without one; those duplicates are ignored.

Features lost or at risk

- **Nothing removed.** GET /api/runs keeps its original group, experiment_id and metadata fields, so existing clients are unaffected.
- **Offline:** with the backend offline, command-line runs are not listed at all (the status line says so). The GUI does not read results/ directly, by design (it never imports the backend).
- **Performance risk:** each listing reads every run's config.yaml, runtime.json and metrics.csv. With hundreds of runs or very large metrics files this could take noticeable time. It was not measured at scale.
- **Multi-method runs** (e.g. PPO + IQL in one experiment) show values from their first agent only, the one whose curves the Monitor shows. The Method column lists all methods.
- **Loading a command-line run into the builder** works only for single-method CartPole/PPO runs, and values the builder does not have (e.g. paradigm or model) are not carried over.
- **Notes:** the adoption path exists, but main currently has no notes editor (save_notes does nothing), so in practice listed runs are never saved to the run store.

Feature 2 — Sort by column

The fix. Sorting is switched on for the table. Each cell is a SortableItem carrying the real value it displays, so '1,000' sorts after '200', '1 h 02 min' after '14.8 s', and '—' stays last in both directions. The table turns sorting off while it is refilled (otherwise rows re-sort mid-fill) and back on afterwards.

File	Code changed	What it does
frontend/widgets.py	SortableItem (16–35)	QTableWidgetItem subclass comparing a stored key; missing keys last in either direction.
frontend/app.py	Results panel setup (355–)	setSortingEnabled(True); default sort by Started, descending.
frontend/app.py	refresh_runs (2042–)	Sorting suspended during the fill; the user's selection is restored after each refresh.

Behaviour that may have changed

- **Row order** follows the clicked header and persists across refreshes (but not across app restarts).
- **Selection is now kept across refreshes.** Before, every refresh of the table (for example when a run finishes or is queued) cleared the table selection. This was needed because the disk listing arrives a moment after start-up and would otherwise drop the user's selection.

Features lost or at risk

- None found. Selection, Load saved config, Compare two runs and View plot all read the run ID stored in column 0, which still holds it. Selection and Load saved config were exercised in the end-to-end test; Compare two runs and View plot were not clicked in testing.

Feature 3 — More useful columns

The fix. Column values are computed by one pure function, result_row(), which returns display text and a sort key for each column. Timing for app-launched runs also comes from the disk listing, because the GUI never recorded start or finish times itself.

File	Code changed	What it does
frontend/model.py	RESULT_COLUMNS, SOURCE_LABELS (137–139)	Column list and labels.
frontend/model.py	result_row, format_duration, _timestamp (204–258)	Text and sort key per column; durations as '14.8 s', '1 min 15 s', '1 h 02 min'; times in local time.
frontend/app.py	Results panel setup (355–), refresh_runs	Eleven columns; Experiment stretches, the rest fit their contents.

Behaviour that may have changed

- **'Reward' became 'Latest reward'** (same value) and moved from column 4 to column 10; 'Best reward' is new.
- **Duration** is blank ('—') for app-launched runs until the backend lists their folder, and always blank for simulated/example runs, which have no training time.
- **Started** is the record's creation time for app runs and the training start time for command-line runs (shown in local time).
- **The table is wider:** on narrow windows the Experiment column gets less room and a horizontal scrollbar can appear.

Features lost or at risk

- Anything that read the table by column position other than column 0 would break. A search of the repository found no such code (all callers read column 0), but external scripts would need updating.

Feature 4 — Hide example runs and filter

The fix. Example runs are hidden once any real run exists; a 'Show example runs' checkbox (shown only then) brings them back. New Source and Status dropdowns filter the table, and the search box now matches the text of every column. The filter rule is one pure function, `matches_filters()`.

File	Code changed	What it does
frontend/model.py	SOURCE_FILTERS, STATUS_FILTERS, STATUS_GROUPS, <code>matches_filters</code> (141–147, 260–271)	Filter choices and the rule that applies them.
frontend/app.py	Results panel setup (355–)	Search box, two dropdowns, checkbox, Refresh button and status line.
frontend/app.py	<code>refresh_runs</code> (2042–)	Applies filters; examples rule; choosing 'Simulated & examples' always shows examples.

Behaviour that may have changed

- **Example runs disappear** from the Results table as soon as a real run exists (they remain in the Monitor dropdown and Plot viewer).
- **Search matches more:** it now also matches method, environment, timesteps, dates, rewards and source, so a short query such as '7' can match more rows than before.

Features lost or at risk

- None removed. Filters reset when the app restarts (they are not saved).

Earlier change on this branch — Results tab visible by default

The fix. "results" added to the default visible list in `frontend/settings.py` (line 81, inside the default `settings.toml` text).

Behaviour that may have changed

- New installs show Results directly under Monitor. Existing installs keep their own settings.toml, so they must switch it on once (Settings & About → Results browser).

Features lost or at risk

- None.

4. Testing

New automated tests (all passing):

Suite	Environment	Tests	Covers
tests/test_api.py	backend venv	3 new (12 total)	/api/runs summaries, unfinished and unreadable folders, version_10 vs version_9.
tests/test_results_model.py	backend venv (no Qt)	7	Run records from disk, row text and sort keys (identical before/after metrics load), durations, filters, metrics parsing.
tests/test_results_browser.py	frontend venv (Qt)	6	Offline state, listing, examples rule, linking app runs (no duplicates, duration filled), numeric sort with missing values last, re-listing keeps records, filters and search, selection across refresh.
tests/test_interactive_chart.py	frontend venv	3 (unchanged)	Unaffected; still passing.

End-to-end check against the real backend

The real FastAPI backend was started on a temporary results folder holding three command-line-style runs (finished CartPole, unfinished CartPole, finished non-CartPole). A script drove the real ThetaIDE window: **29 checks passed, 0 failed**, in two consecutive runs. Checks:

- backend connects and lists results/logs
- table lists the 3 command-line runs
- disk status line
- examples hidden once real runs exist
- Show example runs adds them back
- columns
- method / environment / seed
- timesteps and duration from disk
- best vs latest reward
- long duration format
- unfinished run: status incomplete, no duration
- source label
- default sort: newest first
- sort by best reward (desc): best run first
- sort by duration (asc): numeric, missing last
- sort by duration (desc): missing still last
- status filter: failed or stopped
- source filter: trained in the app (none)
- source filter: simulated & examples shows examples
- search matches environment column
- selection picks the run
- metrics load on selection
- row values unchanged once metrics load (table agrees with chart)
- monitor chart gets the curve
- selection kept across refresh
- saving adopts the run into the store

- adopted run is no longer transient
- re-listing does not duplicate the adopted run
- non-CartPole run is not loaded into the builder

Regression: every existing suite, unmodified HEAD vs. this change

Each suite was run on branch HEAD (in a separate worktree) and on the working tree, with a hard time limit per run.

Suite	Branch HEAD (unmodified)	With these changes
backend	164 passed, 1 warning in 74.09s (0:01:14)	174 passed, 1 warning in 61.16s (0:01:01)
test_experiment_pane	Ran 19 tests in 8.627s FAILED (failures=2) failing: test_select_file_emits_signal test_selecting_tree_item_loads_into_viewer	Ran 19 tests in 12.463s FAILED (failures=2) failing: test_select_file_emits_signal test_selecting_tree_item_loads_into_viewer
test_frontend_layout	Ran 13 tests in 11.041s FAILED (failures=5, errors=1) failing: test_cannot_hide_all_panes test_default_tab_order test_pane_sliders_registered test_reset_sidebar_layout test_save_and_load_layout test_toggle_slider_hides_pane	Ran 13 tests in 11.994s FAILED (failures=3, errors=1) failing: test_cannot_hide_all_panes test_pane_sliders_registered test_save_and_load_layout test_toggle_slider_hides_pane
test_hotkeys	Ran 23 tests in 64.174s OK	Ran 23 tests in 65.287s OK
test_hub_client	Ran 6 tests in 0.056s FAILED (errors=2) failing: test_checksum_mismatch_fails_safely test_download_sha256_verify_and_install	Ran 6 tests in 0.085s FAILED (errors=2) failing: test_checksum_mismatch_fails_safely test_download_sha256_verify_and_install
test_interactive_chart	Ran 3 tests in 0.010s OK	Ran 3 tests in 0.008s OK
test_obsidian_plugin	Ran 4 tests in 0.003s OK (skipped=4)	Ran 4 tests in 0.003s OK (skipped=4)
test_plugin_system	Ran 9 tests in 77.174s OK	Ran 9 tests in 91.141s FAILED (failures=2) failing: test_plugin_disabled_on_initialization test_settings_toggle_slider_interaction
test_results_browser	(not present)	Ran 6 tests in 8.520s OK
test_run_navigator	Ran 7 tests in 9.137s OK	Ran 7 tests in 9.850s OK
test_terminal	Ran 11 tests in 0.188s FAILED (failures=1, skipped=5) failing: test_default_shell_resolution	Ran 11 tests in 0.145s FAILED (failures=1, skipped=5) failing: test_default_shell_resolution

Suites that share settings through the temp folder, rerun in isolation

test_frontend_layout and test_plugin_system create the app with a data folder directly inside the system temp folder, so they read and write one shared %TEMP%\settings.toml. Their whole-suite results depend on what earlier runs left there: in the table above, test_plugin_system failed two tests on the second (changed) run after the first run had left sample_plugin enabled in that file. Rerun with a fresh private temp folder per run, the same two plugin tests failed on unmodified HEAD instead and passed with the changes, so this suite is order- and timing-dependent on both versions and its two failures are not attributable to this change. test_frontend_layout, run one test per process, fails exactly the same four tests on both versions:

```
test_plugin_system [main]: test_plugin_disabled_on_initialization test_settings_toggle_slider_interaction
test_frontend_layout per test [main]: failing -> test_pane_sliders_registered test_toggle_slider_hides_pane
test_cannot_hide_all_panes test_save_and_load_layout
test_plugin_system [change]: OK
test_frontend_layout per test [change]: failing -> test_pane_sliders_registered test_toggle_slider_hides_pane
test_cannot_hide_all_panes test_save_and_load_layout
```

test_plugin_system repeated twice more per version (fresh private temp folder each run)

Across all four runs per version, the same two tests failed in 3 of 4 runs on unmodified HEAD and in 2 of 4 runs with the changes; no other test in the suite failed. The suite is unreliable on both versions, so these failures are not evidence for or against this change.

```
run 1 [main]: test_plugin_disabled_on_initialization test_settings_toggle_slider_interaction
run 1 [change]: OK
run 2 [main]: test_plugin_disabled_on_initialization test_settings_toggle_slider_interaction
run 2 [change]: test_plugin_disabled_on_initialization test_settings_toggle_slider_interaction
```

Not tested

- A run actually launched from the command line during the test (the test data were copies of a real run).
- Large results folders (hundreds of runs) — listing speed is unmeasured.
- Multi-method experiments and SLURM output folders.
- Light themes (the panel uses existing widgets and theme colours only).

5. Pre-existing issues observed (not caused by this change)

- **test_frontend_layout** fails four tests on unmodified HEAD: they expect every panel visible by default, and they share %TEMP%\settings.toml between tests.
- **test_experiment_pane** fails two tests on unmodified HEAD (test_select_file_emits_signal, test_selecting_tree_item_loads_into_viewer).
- **test_hub_client** has two errors and **test_terminal** one failure (test_default_shell_resolution) on unmodified HEAD.
- **Shared test state**: several Qt suites write %TEMP%\settings.toml (sidebar layout, enabled plugins), so their results depend on the order and history of test runs.
- **Exit code 127**: ThetaIDE processes exit with code 127 on shutdown, on unmodified HEAD as well.
- **Monitor storage label** always shows results/jobs/jobs.db, including for simulated and command-line runs.
- **test_hotkeys hung once** (over 30 minutes, on unmodified HEAD) when the whole suite was run with an empty, private temp folder; with the normal temp folder it passes in about 70 seconds on both versions.

Appendix A — Every changed region

Generated from `git diff -u0` against 4f8a5ef fix(plugins): eliminate legacy uninstalled tracking and prevent crash on Hub install. Line ranges are in the working-tree file; the function is the definition enclosing the first changed line.

File	Lines	+ / -	Inside
frontend/app.py	25–27	+3 / -2	(module level)
frontend/app.py	31	+1 / -1	(module level)
frontend/app.py	90–92	+3 / -0	<code>__init__</code>
frontend/app.py	360–362	+3 / -1	<code>make_center</code>
frontend/app.py	364	+1 / -1	<code>make_center</code>
frontend/app.py	366–386	+21 / -3	<code>make_center</code>
frontend/app.py	388–389	+2 / -1	<code>make_center</code>
frontend/app.py	1367–1371	+5 / -0	<code>set_backend_state</code>
frontend/app.py	1450–1452	+3 / -0	<code>persist</code>
frontend/app.py	1457–1458	+2 / -0	<code>persist</code>
frontend/app.py	1746	+1 / -1	<code>open_job</code>
frontend/app.py	1809–1810	+2 / -0	<code>finish</code>
frontend/app.py	1836	+1 / -0	<code>select_run</code>
frontend/app.py	1915	+1 / -10	<code>backfill_received</code>
frontend/app.py	1930–1935	+6 / -1	<code>render_run</code>
frontend/app.py	1937	+1 / -1	<code>render_run</code>
frontend/app.py	2009–2011	+3 / -0	<code>render_run</code>
frontend/app.py	2047–2053	+7 / -0	<code>refresh_runs</code>
frontend/app.py	2055	+1 / -0	<code>refresh_runs</code>
frontend/app.py	2058–2063	+6 / -3	<code>refresh_runs</code>
frontend/app.py	2067–2068	+2 / -5	<code>refresh_runs</code>
frontend/app.py	2071–2076	+6 / -0	<code>refresh_runs</code>
frontend/app.py	2078	+1 / -0	<code>refresh_runs</code>
frontend/app.py	2113–2177	+65 / -0	<code>adds request_disk_runs, disk_runs_received, update_disk_status, load_disk_metrics (+1 more)</code>
frontend/app.py	2197–2198	+2 / -0	<code>table_selected</code>
frontend/app.py	2224–2227	+4 / -0	<code>load_selected_config</code>
frontend/model.py	8	+1 / -0	(module level)
frontend/model.py	111–124	+14 / -0	<code>adds metric_points_from_api</code>
frontend/model.py	135–273	+139 / -0	<code>adds run_name, run_source, disk_run_id, _value (+5 more)</code>
frontend/settings.py	81	+1 / -0	(module level)
frontend/widgets.py	5	+1 / -1	(module level)
frontend/widgets.py	16–37	+22 / -0	<code>adds SortableItem, __init__, __lt__</code>
src/app/api/app.py	113–189	+77 / -0	<code>adds _latest_version_file, number, _float, _agent_summary (+1 more)</code>

File	Lines	+ / -	Inside
src/app/api/app.py	192–196	+5 / -0	list_runs
src/app/api/app.py	210–211	+2 / -0	list_runs
src/app/api/app.py	217–219	+3 / -0	list_runs
src/app/api/app.py	227–228	+2 / -7	get_metrics
tests/test_api.py	85–146	+62 / -0	adds _write_run, test_list_runs_summarizes_each_run, test_list_runs_unfinished_and_unreadable, test_newest_version_is_chosen_by_number
tests/test_results_model.py	1–95	+95 / -0	(new file)
tests/test_results_browser.py	1–105	+105 / -0	(new file)